OPERATING SYSTEMS

Lab practical and viva preparation | Semester 5

300 questions and answers

100 basic | 140 intermediate | 60 expert

17 practical topics | Corrected C programs | Worked traces

Prepared for your uploaded OS lab archives and 48-page handwritten practical report. Learn what each algorithm does, demonstrate a small input, explain the variables, and defend the assumptions behind your output.

This is an original syllabus-based study bank, not a promise of every possible examiner question. The examples distinguish the uploaded code from the corrected companion implementations.

Your fastest preparation route

Time	What to study
15 min	Read the corrections. Memorize the scheduling formulas and the four deadlock conditions.
45 min	FCFS, SJF, RR, Priority and SRTF: draw timelines and calculate WT/TAT/RT.
35 min	First/Best/Worst Fit, MFT and MVT: declare the allocation model before calculating.
30 min	FIFO, LRU and Optimal: trace every hit and fault.
30 min	Banker: calculate Need, show a safe sequence, and explain an unsafe request.
30 min	Prevention, Dining Philosophers and Producer-Consumer: explain lock/semaphore order.
25 min	Compile two randomly chosen programs, change an input, explain the output aloud.

Only one hour? Read the correction pages, the 17 practical cards and the final revision tables. For each experiment rehearse: aim -> data structures -> selection rule -> trace -> limitation.

Scope and how to use this pack

The newer archive lists 12 core experiments. The older archive adds Worst Fit and preemptive SJF/SRTF. The handwritten report adds prevention, Dining Philosophers and Producer-Consumer. Together these establish 17 distinct practical topics; instructor numbering differs between sources.

Group	Topics
CPU scheduling	FCFS, nonpreemptive SJF, Round Robin, nonpreemptive Priority, SRTF
Memory allocation	First Fit, Best Fit, Worst Fit, MFT, MVT
Page replacement	FIFO, LRU, Optimal
Resources and concurrency	Banker, deadlock prevention, Dining Philosophers, Producer-Consumer

Files included

OS-Viva-Questions-Answers.pdf

This guide: practical cards, 300 questions, worked calculations and revision sheets.

os-incourse-lab-viva.html

All 300 questions in a searchable, filterable page. PDF links use the exact filename above. Keep both files in the same folder.

os-viva-code/

17 commented C files with examples, input conventions and a README. Corrected teaching implementations; not literal transcriptions of the uploaded code.

OS-Lab-C-Programs.zip

A convenient download of the code folder, also linked from the HTML page.

What counts as basic, intermediate or expert?

Basic questions establish definitions and direct behavior. Intermediate questions ask you to trace code and explain equations. Expert questions challenge assumptions, edge cases and design tradeoffs. Each of the 20 question sections has 5 basic, 7 intermediate and 3 expert questions.

General OS/process/virtual-memory sections support follow-up questions. Study the assigned practicals before spending time on broader theory. The scanned report page references in the cards refer to PDF positions, not handwritten numbering.

Corrections before you memorize the code

Arrival-time assumptions

The main four scheduling programs treat every arrival as zero. Thus CT=TAT there. For arbitrary arrivals, use TAT=CT-AT and WT=TAT-BT in this single-burst/no-I/O model. A one-time sort cannot correctly handle all arrival-dependent SJF or SRTF cases.

Two different memory-allocation models

Some ZIP programs reserve an entire block for each admitted process. Scanned Best/Worst Fit examples subtract the request and reuse the remainder. Both are teachable models, but their answers differ. The companion fit programs default to whole blocks and accept --split for reusable holes.

Unused memory is not always fragmentation

Internal fragmentation is unused space inside an allocated unit. External fragmentation means scattered free holes can block a contiguous request despite adequate total free space. A simple MVT remaining-memory counter does not demonstrate scattered holes.

One-based indices can exceed the array

An array a[20] has indices 0..19. Several supplied loops use 1..n with fixed arrays, making the declared maximum unsafe. The corrected versions validate counts and use zero-based storage while displaying one-based PIDs.

Avoid magic sentinels and missing failures

The older SRTF code uses slot 9 as a sentinel, colliding with a tenth real process. Fit programs use arbitrary large values or fail to print later results after one rejection. Use chosen=-1 and report every request explicitly.

Priority convention and tie rules

The corrected priority example uses lower number=higher priority, nonpreemptive execution and input-order ties. The original exchange sorts are not always stable for equal keys. State the convention rather than assuming it.

Safety, prevention and concurrency corrections

A safety test is not deadlock prevention

The handwritten pp.38-40 are labelled prevention, but the resource-claim calculation is a safety check. Banker is avoidance: do not enter an unsafe state. Prevention structurally breaks at least one necessary condition. The companion prevention example imposes one global resource order.

Unsafe does not mean already deadlocked

A safe state has at least one completion order under declared maximum claims. An unsafe state lacks that guarantee. Actual deadlock requires a current situation in which the involved tasks cannot progress.

Safety checking is not the whole request algorithm

A complete Banker request check validates Request<=Need and Request<=Available, tries the allocation, checks safety and rolls back if unsafe. The corrected program demonstrates both safety and one optional request.

Release Allocation, not Max

When simulating a process finishing in the safety test, add its currently allocated resources to Work. It does not return resources it never held. Compare each resource type separately.

Sleeping is not synchronization

The Dining Philosophers code should not sleep or block while holding the state mutex. On release, set THINKING and test neighbors. Preventing adjacent eating does not prove starvation freedom.

Finite thread demos need a completion plan

The corrected Producer-Consumer example balances 25 insertions with 25 removals. All threads are joined before semaphores and mutexes are destroyed. Output order varies; the counts and invariants are the checks.

Correct thread function signatures

A pthread start routine returns void* and takes void*. Pass stable argument storage, not the address of a loop variable that is continually changing. Check API return values rather than hiding prototype errors with casts.

Practical 01 | FCFS

Core list; handwritten pp.1-3

Aim and principle

Run ready processes in arrival order. In this program all arrivals are zero, so input order is the queue order.

Algorithm you should explain

Read bursts; start time is the accumulated clock; set CT=start+BT; compute TAT, WT and RT; preserve process identifiers.

Worked check

BT=24,3,3. Timeline: P1 [0,24), P2 [24,27), P3 [27,30). WT=0,24,27; average WT=17; average TAT=27.

Common trap

A long job at the front delays short jobs: the convoy effect. FCFS is nonpreemptive. With nonzero arrivals, start=max(previous completion,arrival).

Examiner follow-up

Explain why WT is not CT and why RT=WT here.

Run, change, explain

Companion file: os-viva-code/fcfs.c. Read the README input order and examples/fcfs.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 04 gives related definitions and follow-ups.

Practical 02 | SJF (nonpreemptive)

Core list; handwritten pp.4-6

Aim and principle

Choose the shortest burst among ready jobs, then let it finish. The supplied classroom version assumes all jobs arrive together.

Algorithm you should explain

Sort records by burst while carrying original PID. Use stable ties. Accumulate completion times and map results back to each PID.

Worked check

BT=6,8,7,3. Order P4,P1,P3,P2. Intervals [0,3),[3,9),[9,16),[16,24). Average WT=7; average TAT=13.

Common trap

A global sort is insufficient for arbitrary arrivals: only ready jobs are eligible. Long jobs can starve if short jobs keep arriving.

Examiner follow-up

Why can SJF minimize average waiting in the all-ready, known-burst model without being universally best?

Run, change, explain

Companion file: os-viva-code/sjf.c. Read the README input order and examples/sjf.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 05 gives related definitions and follow-ups.

Practical 03 | Round Robin

Core list; handwritten pp.7-9

Aim and principle

Give each ready process at most one positive time quantum, returning unfinished processes to the tail.

Algorithm you should explain

Keep remaining[] separate from original bursts. Run min(remaining,q); advance clock; record first start once; set completion only when remaining reaches zero.

Worked check

BT=24,3,3; q=4. Timeline P1 0-4, P2 4-7, P3 7-10, P1 10-14,14-18,18-22,22-26,26-30. CT=30,7,10; WT=6,4,7; average WT=5.6667.

Common trap

q=0 causes no progress unless rejected. A very large quantum approaches FCFS; a tiny quantum increases switching overhead in a real OS. This simulation charges no switching cost.

Examiner follow-up

Why is P1 response time zero but its total waiting time six?

Run, change, explain

Companion file: os-viva-code/round_robin.c. Read the README input order and examples/round_robin.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 06 gives related definitions and follow-ups.

Practical 04 | Priority scheduling

Core list; handwritten pp.10-12

Aim and principle

Run the highest-priority ready job. The corrected classroom version is nonpreemptive and uses smaller numbers for higher priorities.

Algorithm you should explain

Read burst and priority pairs; stably sort by priority; retain PID; compute metrics from the chosen order.

Worked check

Pairs (BT,priority): (10,3),(1,1),(2,4),(1,5),(5,2). Order P2,P5,P1,P3,P4. Average WT=8.2; average TAT=12.

Common trap

Priority numbering is a convention, not a universal rule. Aging raises the chance of service for jobs waiting too long. Priority inversion is a separate lock-related issue.

Examiner follow-up

What changes if priority scheduling is preemptive and a more urgent job arrives?

Run, change, explain

Companion file: os-viva-code/priority.c. Read the README input order and examples/priority.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 07 gives related definitions and follow-ups.

Practical 05 | First Fit

Core list; handwritten pp.13-15

Aim and principle

Scan blocks in address/list order and choose the first eligible block that can hold the request.

Algorithm you should explain

For each request set chosen=-1; scan blocks; stop at the first fit. Whole-block mode marks it used; --split subtracts request size and keeps the remainder reusable.

Worked check

Blocks=100,500,200,300,600; requests=212,417,112,426. Whole-block choices B2,B5,B3,fail. Split-hole choices B2,B5,B2,fail.

Common trap

Whole-block and split-hole models give different answers. A failed request must not silently allocate block zero or suppress later results.

Examiner follow-up

Why can P3 return to B2 in split mode?

Run, change, explain

Companion file: os-viva-code/first_fit.c. Read the README input order and examples/first_fit.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 09 gives related definitions and follow-ups.

Practical 06 | Best Fit

Core list; handwritten pp.16-18

Aim and principle

Among eligible blocks, choose the smallest one that is large enough.

Algorithm you should explain

Scan all blocks while retaining the best index; no arbitrary 9999 sentinel is needed. Resolve equal sizes by the first block index. Apply either whole-block or split mode.

Worked check

For blocks=100,500,200,300,600 and requests=212,417,112,426, choices are B4,B2,B3,B5 in both modes. Split remainders become 100,83,88,88,174.

Common trap

Small immediate leftover is not a guarantee of best future allocation. Many tiny holes can be hard to reuse. The scanned notebook subtracts remainders, unlike some ZIP programs.

Examiner follow-up

Does best fit always minimize total future waste? Explain with workload dependence.

Run, change, explain

Companion file: os-viva-code/best_fit.c. Read the README input order and examples/best_fit.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 09 gives related definitions and follow-ups.

Practical 07 | Worst Fit

Older archive; handwritten pp.19-21

Aim and principle

Choose the largest currently eligible block or hole, aiming to leave a relatively large remainder.

Algorithm you should explain

Scan all candidates; retain the largest adequate size; use the first index for ties; update allocation according to the selected model.

Worked check

Same blocks and requests: whole-block B5,B2,B4,fail; split-hole B5,B2,B5,fail. Split final free sizes=100,83,200,300,276.

Common trap

In split mode 959 units remain in total, but no single hole fits 426. This is a concrete external-fragmentation example.

Examiner follow-up

Why does a large total of free memory not guarantee one contiguous allocation?

Run, change, explain

Companion file: os-viva-code/worst_fit.c. Read the README input order and examples/worst_fit.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 09 gives related definitions and follow-ups.

Practical 08 | FIFO page replacement

Core list; handwritten pp.22-24

Aim and principle

On a miss with full frames, evict the resident page that entered earliest. Hits do not renew its insertion age.

Algorithm you should explain

Track frames, occupancy and a circular replacement pointer. Search all resident frames for a hit. Advance the pointer only when inserting on a fault.

Worked check

For the 20-reference practice string in the trace section with 3 frames: 15 faults and 5 hits. For 1,2,3,4,1,2,5,1,2,3,4,5: 3 frames gives 9 faults; 4 gives 10.

Common trap

FIFO can exhibit Belady's anomaly. Do not advance its pointer on a hit or interpret a blank trace row as an empty frame.

Examiner follow-up

Why can adding a frame increase FIFO faults?

Run, change, explain

Companion file: os-viva-code/page_fifo.c. Read the README input order and examples/page_fifo.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 12 gives related definitions and follow-ups.

Practical 09 | LRU page replacement

Core list; handwritten pp.25-27

Aim and principle

Evict the resident page whose last use is farthest in the past.

Algorithm you should explain

Update a last-used timestamp on every hit and insertion. On a full-frame miss select the lowest timestamp, not the lowest page number.

Worked check

The shared 20-reference string gives 12 faults and 8 hits with 3 frames. On 1,2,3,1,4 with 3 frames, LRU evicts 2; FIFO evicts 1.

Common trap

Exact LRU maintenance has cost. A simulator timestamp is not a claim that real hardware provides free exact recency information.

Examiner follow-up

Why must the timestamp change on a hit?

Run, change, explain

Companion file: os-viva-code/page_lru.c. Read the README input order and examples/page_lru.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 13 gives related definitions and follow-ups.

Practical 10 | Optimal page replacement

Core list; handwritten pp.28-31

Aim and principle

Evict the resident page whose next reference lies farthest in the future, treating no future use as farthest.

Algorithm you should explain

Search the suffix of the input for each resident page. Choose the farthest next use; use the lowest frame index for ties. Empty frames are filled before any eviction.

Worked check

The shared 20-reference string gives 9 faults and 11 hits with 3 frames. It supplies a lower-bound comparison for these policies on the same fixed string.

Common trap

It needs future knowledge and is primarily a benchmark. Future count is not the same as next-use distance.

Examiner follow-up

If two pages never occur again, can either be chosen?

Run, change, explain

Companion file: os-viva-code/page_optimal.c. Read the README input order and examples/page_optimal.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 14 gives related definitions and follow-ups.

Practical 11 | MFT: fixed partitions

Core list; ZIP MFT materials

Aim and principle

Split memory into a fixed number of equal partitions. Admit at most one process per partition in this simplified model.

Algorithm you should explain

partitions=total/size; tail=total%size. A request must fit one partition and a partition must be available. Sum partition-size minus request only for admitted jobs.

Worked check

Total=1000, partition=300, requests=275,400,290,293,100. Three partitions, tail=100. Admit P1,P3,P4; internal waste=25+10+7=42. P2 too large; P5 has no partition.

Common trap

Distinguish internal waste, unused whole partitions and the tail outside formed partitions. Do not automatically call every leftover external fragmentation.

Examiner follow-up

Why cannot P2 use several separate fixed partitions in this model?

Run, change, explain

Companion file: os-viva-code/mft.c. Read the README input order and examples/mft.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 10 gives related definitions and follow-ups.

Practical 12 | MVT: variable allocation

Core list; ZIP MVT materials

Aim and principle

Allocate requested sizes from available memory. The supplied numerical exercise is a simple free-pool accounting model.

Algorithm you should explain

Keep remaining=total; if request<=remaining subtract it, otherwise reject it. The corrected program continues to consider later requests.

Worked check

Total=1000; requests=400,275,550. First two use 675; 325 remains. The 550 request fails even though the memory is not completely full.

Common trap

A single remaining counter does not represent multiple holes, deallocation or compaction. Real dynamic partitions can suffer external fragmentation.

Examiner follow-up

What additional data structure is needed to represent scattered free holes?

Run, change, explain

Companion file: os-viva-code/mvt.c. Read the README input order and examples/mvt.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 10 gives related definitions and follow-ups.

Practical 13 | Banker's safety and requests

Core list; handwritten pp.32-37

Aim and principle

Avoid unsafe allocations using maximum claims. Need=Max-Allocation. A safe sequence is an order in which every process could finish.

Algorithm you should explain

Copy Available into Work; finish[] starts false. Repeatedly find Need[i]<=Work componentwise; set Work+=Allocation[i], finish[i]=true. A request also needs bounds checks, tentative update, safety test and rollback on failure.

Worked check

Classic matrix in the worked section starts Available=(3,3,2), safe sequence P2,P4,P5,P1,P3. P2 request (1,0,2) is safe. From the original state, P5 request (3,3,0) is unsafe and is rolled back.

Common trap

Unsafe does not mean already deadlocked. Completion releases Allocation, not Max. The file implements a single optional request after the initial safety check.

Examiner follow-up

Why is componentwise comparison necessary rather than comparing resource sums?

Run, change, explain

Companion file: os-viva-code/banker.c. Read the README input order and examples/banker.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 16 gives related definitions and follow-ups.

Practical 14 | SRTF: preemptive SJF

Older archive Lab09; supplemental scheduling practical

Aim and principle

At each decision, run the arrived unfinished job with the shortest remaining burst.

Algorithm you should explain

Keep original burst, remaining, arrival, first-start and completion. If no job is ready, jump to the next arrival. This simple implementation reselects after each unit of CPU service.

Worked check

(AT,BT)=(0,8),(1,4),(2,9),(3,5). Timeline P1 0-1, P2 1-5, P4 5-10, P1 10-17, P3 17-26. CT=17,5,26,10; average WT=6.5; average TAT=13.

Common trap

Avoid the supplied fixed sentinel slot at index 9: it collides with a real tenth process. Explicitly represent no chosen process with -1.

Examiner follow-up

Why is sorting the original bursts once not enough?

Run, change, explain

Companion file: os-viva-code/srtf.c. Read the README input order and examples/srtf.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 05 gives related definitions and follow-ups.

Practical 15 | Deadlock prevention

Handwritten pp.38-40; conceptual correction

Aim and principle

Design the resource protocol so at least one necessary deadlock condition cannot hold. The companion example breaks circular wait by global lock ordering.

Algorithm you should explain

Two threads both need resources 0 and 1. Regardless of their conceptual preference, both acquire 0 then 1, update a shared counter, then release in reverse order. Each performs 1000 iterations.

Worked check

Expected final output: COMPLETED 2000 updates; lock order 0 then 1. No thread can hold resource 1 while waiting to acquire resource 0 in this protocol.

Common trap

The notebook page labelled prevention actually checks remaining claims against available resources, which is a Banker-style safety calculation. State the distinction politely in viva.

Examiner follow-up

Which Coffman condition is broken, and why is this different from avoidance?

Run, change, explain

Companion file: os-viva-code/deadlock_prevention.c. Read the README input order and examples/deadlock_prevention.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 15 gives related definitions and follow-ups.

Practical 16 | Dining Philosophers

Handwritten pp.41-44

Aim and principle

Coordinate five participants sharing adjacent forks so neighbors never eat together.

Algorithm you should explain

Under a mutex, mark HUNGRY and test both neighbors. If eligible, set EATING and post private permission. Wait outside the mutex; after eating set THINKING and test the neighbors.

Worked check

The finite demonstration gives each philosopher three meals and checks a total of 15. Order may differ each run. Nonadjacent philosophers may eat concurrently; at most two can eat with five in a ring.

Common trap

Deadlock freedom does not by itself establish starvation freedom. The example has no fairness queue. Avoid sleeping or waiting while holding the state mutex.

Examiner follow-up

Why does test() reserve the forks before posting permission?

Run, change, explain

Companion file: os-viva-code/dining_philosophers.c. Read the README input order and examples/dining_philosophers.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 19 gives related definitions and follow-ups.

Practical 17 | Producer-Consumer

Handwritten pp.45-48

Aim and principle

Transfer items through a bounded circular buffer without overflow, underflow or duplicate consumption.

Algorithm you should explain

Capacity 5; empty=5, full=0; mutex unlocked. Producer: wait empty, lock, insert, unlock, post full. Consumer: wait full, lock, remove, unlock, post empty.

Worked check

Five producers each create five unique items; five consumers each remove five. Expected totals: produced=25, consumed=25, remaining=0. The order is nondeterministic; every item is checked exactly once.

Common trap

A counting semaphore tracks capacity or items; the mutex protects indices and data. Equal finite totals are essential for this demo to terminate.

Examiner follow-up

Why would lock mutex then wait(full) risk deadlock?

Run, change, explain

Companion file: os-viva-code/producer_consumer.c. Read the README input order and examples/producer_consumer.in where provided. Threaded examples require no input. Full corrected sources are included separately to keep this guide readable.

First reproduce the example. Next change one value and predict the result before running it. Finally explain why the result differs, including one relevant edge case.

Question-bank section 17 gives related definitions and follow-ups.

Worked scheduling comparison

Use the same all-ready workload BT=(6,8,7,3), no I/O and no switching cost. This example illustrates why policy changes alter waiting; it does not claim one policy wins every workload.

Policy	Execution order / slices	Average WT
FCFS	P1 0-6; P2 6-14; P3 14-21; P4 21-24	10.25
SJF	P4 0-3; P1 3-9; P3 9-16; P2 16-24	7.00
RR, q=4	P1 0-4; P2 4-8; P3 8-12; P4 12-15; P1 15-17; P2 17-21; P3 21-24	13.25

FCFS CT=(6,14,21,24), WT=(0,6,14,21). SJF CT=(9,24,16,3), WT=(3,16,9,0). RR CT=(17,21,24,15), WT=(11,13,17,12). Averages are sums divided by 4.

Arrival-aware SRTF trace

PID	AT	BT	First start	CT	TAT	WT	RT
P1	0	8	0	17	17	9	0
P2	1	4	1	5	4	0	0
P3	2	9	17	26	24	15	15
P4	3	5	5	10	7	2	2

At t=1, P1 has 7 remaining and newly arrived P2 needs 4, so preempt P1. At t=5, compare P1 remaining 7, P3 remaining 9 and P4 remaining 5: choose P4. Average WT=26/4=6.5 and average TAT=52/4=13.

Worked memory allocation: declare the model

Blocks/holes in order: B1=100, B2=500, B3=200, B4=300, B5=600. Requests: P1=212, P2=417, P3=112, P4=426. No freeing or coalescing occurs in this exercise.

Policy / mode	P1	P2	P3	P4
First / whole	B2	B5	B3	Fail
First / split	B2	B5	B2	Fail
Best / whole	B4	B2	B3	B5
Best / split	B4	B2	B3	B5
Worst / whole	B5	B2	B4	Fail
Worst / split	B5	B2	B5	Fail

Trace split First Fit

Start: [100,500,200,300,600].

After 212 in B2: [100,288,200,300,600].

After 417 in B5: [100,288,200,300,183].

After 112 in B2: [100,176,200,300,183].

Request 426 fails: total free=959, but the largest hole is only 300.

Code distinction

Whole-block: used[chosen]=1; the unused remainder inside that assigned block is not offered to another process. Split-hole: free_size[chosen]-=request; the remainder is still eligible for future requests. Both approaches require request<=candidate size.

When your examiner says First Fit but does not state the model, clarify whether the remaining hole can be reused. Show that you understand the distinction rather than memorizing a single allocation table.

Page replacement: complete practice trace

Reference string: 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1. Three frames start empty. A frame slot stays in its slot when another slot is replaced. H=hit; F=fault.

Step / page	FIFO frames	LRU frames	OPT frames
1 / 7	7 - - F	7 - - F	7 - - F
2 / 0	7 0 - F	7 0 - F	7 0 - F
3 / 1	7 0 1 F	7 0 1 F	7 0 1 F
4 / 2	2 0 1 F	2 0 1 F	2 0 1 F
5 / 0	2 0 1 H	2 0 1 H	2 0 1 H
6 / 3	2 3 1 F	2 0 3 F	2 0 3 F
7 / 0	2 3 0 F	2 0 3 H	2 0 3 H
8 / 4	4 3 0 F	4 0 3 F	2 4 3 F
9 / 2	4 2 0 F	4 0 2 F	2 4 3 H
10 / 3	4 2 3 F	4 3 2 F	2 4 3 H
11 / 0	0 2 3 F	0 3 2 F	2 0 3 F
12 / 3	0 2 3 H	0 3 2 H	2 0 3 H
13 / 2	0 2 3 H	0 3 2 H	2 0 3 H
14 / 1	0 1 3 F	1 3 2 F	2 0 1 F
15 / 2	0 1 2 F	1 3 2 H	2 0 1 H
16 / 0	0 1 2 H	1 0 2 F	2 0 1 H
17 / 1	0 1 2 H	1 0 2 H	2 0 1 H
18 / 7	7 1 2 F	1 0 7 F	7 0 1 F
19 / 0	7 0 2 F	1 0 7 H	7 0 1 H
20 / 1	7 0 1 F	1 0 7 H	7 0 1 H

Fault ratios: FIFO=15/20=75%; LRU=12/20=60%; Optimal=9/20=45%. Faults plus hits must equal 20.

Banker: matrix and safe sequence

Five processes P1-P5; resource types A,B,C; Available=(3,3,2). Each vector comparison is componentwise.

PID	Allocation	Max	Need=Max-Allocation
P1	0 1 0	7 5 3	7 4 3
P2	2 0 0	3 2 2	1 2 2
P3	3 0 2	9 0 2	6 0 0
P4	2 1 1	2 2 2	0 1 1
P5	0 0 2	4 3 3	4 3 1

Selected	Work before	Need	Work after release
P2	3 3 2	1 2 2	5 3 2
P4	5 3 2	0 1 1	7 4 3
P5	7 4 3	4 3 1	7 4 5
P1	7 4 5	7 4 3	7 5 5
P3	7 5 5	6 0 0	10 5 7

All five finish in the hypothetical sequence: SAFE. The final Work=(10,5,7) equals total resources. A different valid safe sequence may also exist.

Two requests from the original state

P2 requests (1,0,2)

It fits Need=(1,2,2) and Available=(3,3,2). Tentative Available becomes (2,3,0); P2 remaining Need becomes (0,2,0), so P2 can finish and release its updated allocation. A full safe sequence exists: grant.

P5 requests (3,3,0)

It fits Need and Available, but tentative Available becomes (0,0,2). No unfinished process has all components of Need at most this Work. The tentative state is unsafe: roll back. Do not confuse availability with safety.

C code: variables and defense

Family	Variables you must explain
Scheduling	burst=original CPU demand; remaining=unfinished service; clock=simulation time; first=first service time; completion=last service end; PID=identity after sorting.
Fit allocation	free_size=current block or hole capacity; used=whole-block reservation; chosen=-1 means no candidate; --split reuses remainders.
Page replacement	frames=resident page IDs; occupancy=filled slots; FIFO pointer=insertion victim; LRU timestamp=last use; OPT next use=future distance.
Banker	Allocation=current holdings; Max=declared maximum; Need=remaining maximum; Available=unallocated units; Work=temporary safety resources; Finish=hypothetical completion.
Producer-Consumer	empty=available slots; full=available items; in/out=circular indices; mutex=buffer exclusivity; seen[]=test that each item is consumed once.
Dining Philosophers	state[]=thinking/hungry/eating; permission[]=private semaphores; mutex=atomic eligibility checks and transitions.

Build and run

Standard example: gcc -std=c11 -Wall -Wextra -pedantic fcfs.c -o fcfs

Then: ./fcfs < examples/fcfs.in

Threaded example: gcc -std=c11 -Wall -Wextra -pedantic -pthread producer_consumer.c -o producer_consumer

Then: ./producer_consumer

Threaded files use POSIX pthreads and unnamed semaphores and were tested on Linux. Use Linux/WSL or a compatible environment. A plain Windows C compiler is not automatically a POSIX environment. All 17 sources are separate programs; do not link them together, since each defines main.

Five useful oral checks

Why this input bound? What happens on zero or invalid input? What invariant holds after each iteration? Why does the loop terminate? Which host OS behavior is merely being simulated?

Question bank | Topic index

01. Operating-system foundations - Q001-Q015

02. Processes, threads and states - Q016-Q030

03. Scheduling metrics and Gantt charts - Q031-Q045

04. Practical: FCFS scheduling - Q046-Q060

05. Practical: SJF and supplemental SRTF - Q061-Q075

06. Practical: Round Robin - Q076-Q090

07. Practical: Priority scheduling - Q091-Q105

08. Contiguous allocation and fragmentation - Q106-Q120

09. Practical: First, best and worst fit - Q121-Q135

10. Practical: MFT and MVT - Q136-Q150

11. Paging and page faults - Q151-Q165

12. Practical: FIFO page replacement - Q166-Q180

13. Practical: LRU page replacement - Q181-Q195

14. Practical: Optimal replacement - Q196-Q210

15. Deadlock fundamentals - Q211-Q225

16. Practical: Banker's algorithm - Q226-Q240

17. Producer-consumer and synchronization - Q241-Q255

18. Virtual-memory and address-translation follow-ups - Q256-Q270

19. Dining Philosophers - Q271-Q285

20. C code, debugging and practical defense - Q286-Q300

01 | Operating-system foundations

Q001 [Basic] What is an operating system?

System software that manages hardware resources and provides services and abstractions for programs, such as processes, virtual memory and files.

Q002 [Basic] What is the kernel?

The privileged core of an operating system. It manages protected resources and mediates operations that ordinary applications cannot safely perform directly.

Q003 [Basic] What is a system call?

A controlled interface through which a program requests an OS service, such as opening a file or creating a process. It normally involves a transition into kernel execution.

Q004 [Basic] What is multiprogramming?

Keeping multiple jobs available so the CPU can run another when one waits, improving resource utilization. It does not require multiple CPUs.

Q005 [Basic] What is multitasking?

Sharing execution among tasks, often through frequent scheduling and preemption, so several activities make progress over time.

Q006 [Intermediate] Distinguish an OS from a shell.

The OS manages resources and protection. A shell is a user interface and command interpreter that requests OS services; it is not the entire operating system.

Q007 [Intermediate] What is the difference between user mode and kernel mode?

User mode restricts privileged operations. Kernel mode permits protected hardware and memory-management actions. Hardware mechanisms enforce the boundary rather than relying only on application cooperation.

Q008 [Intermediate] What is an interrupt?

An event that transfers control to a handler, often caused asynchronously by hardware. A timer interrupt can let the OS regain control and reconsider scheduling.

Q009 [Intermediate] What is a trap or synchronous exception?

An event caused by the current instruction, such as a system-call instruction, divide error or page fault. Terminology varies by architecture, so distinguish the cause and handling behavior.

Q010 [Intermediate] What is a real-time system?

A system whose correctness depends on meeting timing requirements as well as computing correct results. Hard and soft deadlines have different consequences when missed.

Q011 [Intermediate] What is the difference between concurrency and parallelism?

Concurrency means tasks have overlapping lifetimes or progress. Parallelism means tasks execute simultaneously on separate processing resources. A single CPU can support concurrency by interleaving work.

Q012 [Intermediate] Why are the lab programs called simulations?

They calculate outcomes from input arrays and policy rules. They do not replace the host OS scheduler, allocate physical partitions, or directly evict the host's memory pages.

Q013 [Expert] Does a system call always cause a process context switch?

No. It changes privilege context, but the same process may continue executing and return. A process switch occurs only if scheduling selects a different execution context.

Q014 [Expert] Why should an OS separate mechanism from policy?

Mechanism provides a capability, such as switching contexts; policy decides when and whom to switch. Separation allows policies to change without redesigning every low-level operation.

Q015 [Expert] Why cannot one scheduling algorithm be called universally best?

Workloads and objectives differ: response time, throughput, deadlines, fairness and overhead may conflict. State the assumptions and metric before comparing algorithms.

02 | Processes, threads and states

Q016 [Basic] What is a process?

An executing program with its execution state and resources, including an address space and OS-maintained information. A program file alone is not a running process.

Q017 [Basic] What is a thread?

An execution stream within a process with its own registers and stack. Threads in a process normally share its address space and many resources.

Q018 [Basic] What is a PCB?

A Process Control Block stores information needed to manage a process, such as saved execution state, scheduling information, identifiers and resource references.

Q019 [Basic] Name the common process states.

New, ready, running, blocked or waiting, and terminated. More detailed OS models may include suspended states.

Q020 [Basic] What is the ready queue?

The set or queue of processes eligible to execute but not currently running. A blocked process is waiting for an event and is not ready.

Q021 [Intermediate] What happens during a context switch?

The OS saves the current task's execution state and restores another's. Scheduler work, address-space changes and cache effects can add cost beyond saving registers.

Q022 [Intermediate] What is the difference between ready and blocked?

A ready process only needs CPU service. A blocked process needs an event, such as I/O completion, before it can execute usefully.

Q023 [Intermediate] Explain running-to-ready and running-to-blocked transitions.

Preemption can move a running process to ready. Waiting for I/O or a synchronization condition moves it to blocked. The reason for leaving the CPU matters.

Q024 [Intermediate] What is a CPU burst?

A period of computation between waits such as I/O. The scheduling exercises simplify each job to one known CPU burst unless explicitly stated otherwise.

Q025 [Intermediate] Distinguish CPU-bound and I/O-bound work.

CPU-bound work spends much time computing; I/O-bound work frequently waits for devices. Real applications can move between these behaviors during execution.

Q026 [Intermediate] What does a dispatcher do?

It gives control of the CPU to the selected task, including the necessary context and execution-mode changes. The scheduler chooses; the dispatcher implements the transfer.

Q027 [Intermediate] What resources do threads share and not share?

Threads normally share code, heap and open resources within a process, but have separate stacks and register state. Shared memory creates synchronization requirements.

Q028 [Expert] Why does a ready queue not always mean a simple FIFO data structure?

A policy may require priorities, multiple queues, trees or other ordering structures. The term describes eligible work, not a mandatory implementation.

Q029 [Expert] Can a blocked process become running immediately after I/O completion?

Usually it becomes ready first, then runs when scheduled. It may quickly preempt another task under the chosen policy, but readiness and execution remain distinct states.

Q030 [Expert] Why can adding threads fail to improve speed?

A workload may be serial, lock-bound, memory-bound or limited by available cores. Scheduling, synchronization and communication overhead can outweigh useful parallel work.

03 | Scheduling metrics and Gantt charts

Q031 [Basic] What is arrival time, AT?

The time a process becomes available to the scheduler in the model. If all arrivals are assumed zero, explicitly say so before calculating results.

Q032 [Basic] What is burst time, BT?

The CPU service required by a process in the simplified exercise. It does not include time spent waiting in the ready queue.

Q033 [Basic] What is completion time, CT?

The clock time when the process finishes its final CPU service. For preemptive scheduling it is the end of the last execution slice.

Q034 [Basic] What is turnaround time?

TAT = CT - AT. It measures elapsed time from arrival to completion, including waiting and all other modeled delays.

Q035 [Basic] What is response time?

RT = first CPU start time - AT. It measures time until the first service, not time until the whole job finishes.

Q036 [Intermediate] How is waiting time calculated in these lab models?

WT = TAT - BT, because the model has one CPU burst and no I/O blocking. With additional blocked time, subtract that too when measuring ready-queue waiting.

Q037 [Intermediate] Why is CT equal to TAT in many supplied programs?

They assume AT=0 for every process. The equality is a consequence of that assumption, not the general definition of turnaround time.

Q038 [Intermediate] When are response time and waiting time equal?

In a nonpreemptive single-burst model, a process waits once before starting, so RT=WT. With preemption it can wait again, making total WT larger.

Q039 [Intermediate] What is a Gantt chart?

A timeline of which process uses the CPU over each interval. Include idle intervals and any context-switch cost if the question specifies them.

Q040 [Intermediate] How are average waiting and turnaround times found?

Add the per-process values and divide by the number of processes. Convert to floating-point before division when a fractional result is possible.

Q041 [Intermediate] What are throughput and CPU utilization?

Throughput is completed jobs per unit time. CPU utilization is the fraction of the stated observation interval spent busy; clarify whether overhead counts as useful work.

Q042 [Intermediate] How do you check a scheduling answer quickly?

Ensure each process receives exactly its burst time, no process runs before arrival, intervals do not overlap on one CPU, and CT, TAT, WT and RT follow their definitions.

Q043 [Expert] What does a negative waiting time usually indicate?

An invalid input or a calculation error, such as subtracting burst time from the wrong completion time or scheduling a job before arrival. It is not a legitimate result in these models.

Q044 [Expert] Why can the same workload produce different valid Gantt charts?

Equal priorities or burst lengths need a tie rule, and simultaneous arrival/quantum events need an ordering rule. State the convention before comparing outputs.

Q045 [Expert] Does low average waiting time guarantee fairness?

No. An average can hide a process waiting extremely long. Inspect worst-case waiting, starvation behavior and response time as well.

04 | Practical: FCFS scheduling

Q046 [Basic] What does FCFS stand for?

First Come First Served. Processes are selected in arrival order; for equal arrivals the lab uses input order.

Q047 [Basic] Is the usual FCFS CPU policy preemptive?

No. Once selected, a process retains the CPU until it completes its current CPU burst or blocks.

Q048 [Basic] What is the aim of the FCFS practical?

Simulate execution order and calculate each process's waiting and turnaround times, then their averages.

Q049 [Basic] What is the initial waiting time with all arrivals zero?

The first selected process has waiting time zero, assuming the CPU is initially available and there is no setup overhead.

Q050 [Basic] What is the zero-arrival FCFS recurrence?

WT[0]=0 and WT[i]=WT[i-1]+BT[i-1]. Then TAT[i]=WT[i]+BT[i]. These recurrences require the stated ordering and arrival assumptions.

Q051 [Intermediate] Trace bursts 24,3,3 in input order.

P1 runs 0-24, P2 24-27 and P3 27-30. Waiting times are 0,24,27; turnaround times are 24,27,30.

Q052 [Intermediate] What are the averages for that FCFS example?

Average WT=(0+24+27)/3=17. Average TAT=(24+27+30)/3=27 time units.

Q053 [Intermediate] What is the convoy effect?

Short or I/O-oriented tasks can queue behind a long CPU burst, delaying their progress and potentially reducing balanced use of CPU and devices.

Q054 [Intermediate] How should FCFS handle different arrival times?

Order by arrival, breaking ties consistently. Start each job at max(previous completion, its arrival), then calculate CT and TAT from that actual start.

Q055 [Intermediate] Give an FCFS example with an idle gap.

P1 has AT=2,BT=3 and P2 AT=10,BT=2. CPU is idle 0-2 and 5-10; execution is P1 2-5 and P2 10-12. Both waiting times are zero.

Q056 [Intermediate] Why preserve process identifiers?

The output must report the correct result for each original process, especially after sorting. An array position is not necessarily the process identity.

Q057 [Intermediate] What is the time complexity of the supplied zero-arrival calculation?

O(n) once the order is known. Sorting unsorted arrivals would add the complexity of the chosen sorting method.

Q058 [Expert] Can ordinary FCFS starve a job already in a finite queue?

With finite service times and no earlier task that never yields or finishes, it eventually runs. It can still experience very long waiting.

Q059 [Expert] Why is changing only the burst list insufficient to support arrivals?

The algorithm must consider eligibility and CPU idle time. Prefix sums alone incorrectly assume continuous availability of every process from time zero.

Q060 [Expert] Why does a successful run not prove the source array bounds are safe?

A small sample may avoid the error. An array of length 20 has indices 0-19; using indices 1 through 20 writes past its end even if smaller examples work.

05 | Practical: SJF and supplemental SRTF

Q061 [Basic] What is SJF?

Shortest Job First selects the ready process with the smallest next CPU burst. The core lab version is nonpreemptive.

Q062 [Basic] What is SRTF?

Shortest Remaining Time First is a preemptive variant that selects the ready process with the least remaining CPU service.

Q063 [Basic] How is zero-arrival nonpreemptive SJF implemented simply?

Sort processes by increasing burst time, use a clear tie rule, then accumulate execution time as in FCFS.

Q064 [Basic] What information does SJF need?

The next CPU burst length or an estimate. The lab reads exact values, whereas a real scheduler generally lacks perfect future knowledge.

Q065 [Basic] What is SJF's main starvation concern?

A long waiting job may repeatedly lose to arriving short jobs. Aging or another fairness mechanism can reduce this risk.

Q066 [Intermediate] Trace SJF for bursts 6,8,7,3.

Order is P4,P1,P3,P2. Execution intervals are 0-3,3-9,9-16,16-24. Waiting in that order is 0,3,9,16.

Q067 [Intermediate] Find SJF averages for 6,8,7,3.

Total WT=28, so average WT=7. Total TAT=3+9+16+24=52, so average TAT=13.

Q068 [Intermediate] Why is a global burst sort wrong when arrivals differ?

A short process may not yet exist in the ready set. At each dispatch choose the shortest eligible job, or explicitly move time forward if no job is ready.

Q069 [Intermediate] When should SRTF preempt the current process?

When an eligible process has a shorter remaining service than the current one, subject to the stated tie rule. Compare remaining time, not original burst time.

Q070 [Intermediate] What are the results for SRTF jobs (AT,BT)=(0,8),(1,4),(2,9),(3,5)?

The schedule is P1 0-1, P2 1-5, P4 5-10, P1 10-17, P3 17-26. CT is 17,5,26,10 in original process order.

Q071 [Intermediate] Find waiting times for that SRTF example.

TAT=17,4,24,7. Subtract BT=8,4,9,5 to get WT=9,0,15,2. Average WT=6.5 and average TAT=13.

Q072 [Intermediate] How should an SRTF simulator represent an idle CPU?

Use an explicit 'no selected process' state and advance to the next arrival. Do not decrement a fake process's burst as a substitute for idle time.

Q073 [Expert] Under what assumptions does SJF minimize mean waiting time?

For a fixed set of jobs ready together, known burst lengths and no preemption cost, shortest-first order minimizes total waiting. Do not generalize the simple proof to every online or constrained setting.

Q074 [Expert] What is wrong with using array element 9 as a 9999 sentinel in the older SRTF code?

It overwrites a real tenth process, assumes real bursts stay below the sentinel and may decrement the sentinel during idle time. Explicit selection state avoids these errors.

Q075 [Expert] Can the original exchange-sort code reverse equal-burst processes?

Yes. Even if it swaps only when one burst is greater, nonadjacent swaps can move equal keys past each other. A stable sort or explicit secondary key is needed for a guaranteed FCFS tie rule.

06 | Practical: Round Robin

Q076 [Basic] What is Round Robin?

A preemptive policy that cycles through the ready queue, allowing each task at most a time quantum before requeueing it if unfinished.

Q077 [Basic] What is the time quantum?

The maximum CPU service allowed in one turn before time-based preemption. It must be positive in this simulation.

Q078 [Basic] What does remaining burst time store?

CPU service still required by the process. Keep the original burst separately for waiting-time calculations.

Q079 [Basic] When does a process leave the RR queue permanently?

When its remaining burst reaches zero. A completed process must not receive another CPU slice.

Q080 [Basic] What happens when remaining burst is less than the quantum?

Run only for the remaining service, finish the process and give the CPU to the next ready process without wasting the unused quantum.

Q081 [Intermediate] Trace bursts 24,3,3 with quantum 4.

P1 runs 0-4, P2 4-7, P3 7-10, then P1 10-14,14-18,18-22,22-26,26-30. The final repeated P1 slices have no competing ready job.

Q082 [Intermediate] What are RR metrics for that example?

CT=TAT=30,7,10. WT=6,4,7 and RT=0,4,7. Average WT=17/3=5.666667; average TAT=47/3=15.666667.

Q083 [Intermediate] What happens when the quantum is very large?

With all processes initially ready and quantum at least every burst, RR behaves as FCFS in the initial queue order.

Q084 [Intermediate] What happens when the quantum is very small?

Tasks get frequent service, but real context-switch overhead can become large. The classroom simulation usually ignores that cost.

Q085 [Intermediate] Why is a simple repeated array scan valid in the supplied RR example?

All jobs arrive at zero and start in a fixed circular order. With later arrivals, a proper ready-queue insertion rule is required.

Q086 [Intermediate] Why is zero quantum dangerous?

Remaining work never decreases, so the loop may never terminate. Reject zero or negative quantum before entering the simulation.

Q087 [Intermediate] When should CT be recorded in RR?

At the instant a process's remaining burst becomes zero. Intermediate end-of-slice times are not its final completion time.

Q088 [Expert] Does q=1 make RR exactly SRTF?

No. RR still rotates fairly through eligible jobs; SRTF repeatedly chooses the smallest remaining work. They can produce different orders even with unit-time simulation.

Q089 [Expert] What must be specified for an arrival exactly at a quantum boundary?

Whether that arrival is enqueued before or after the just-preempted process. Either consistent convention may be used in an exercise, but results can differ.

Q090 [Expert] How do you estimate quantum overhead?

If every full quantum q is followed by switch cost s, useful fraction is approximately q/(q+s) under those assumptions. Final slices, idle intervals and same-process continuation can change the actual result.

07 | Practical: Priority scheduling

Q091 [Basic] What is priority scheduling?

Select the eligible process with the highest scheduling priority. The core supplied program is nonpreemptive and assumes all arrivals zero.

Q092 [Basic] Does a smaller numeric value always mean higher priority?

No universal convention exists. The supplied program sorts ascending, so a smaller number means higher priority in this guide.

Q093 [Basic] Can priority scheduling be preemptive?

Yes. A newly ready higher-priority process may preempt the current one in a preemptive variant. The nonpreemptive variant waits for the current burst to end.

Q094 [Basic] What is aging?

Gradually improving a waiting process's priority so low-priority work is less likely to starve.

Q095 [Basic] What data must stay together when sorting?

Process identity, burst time and priority, plus any other attributes. Sorting only priority values destroys the relationship between jobs and their properties.

Q096 [Intermediate] Trace the supplied pairs (BT,priority)=(10,3),(1,1),(2,4),(1,5),(5,2).

Order is P2,P5,P1,P3,P4. Intervals are 0-1,1-6,6-16,16-18,18-19.

Q097 [Intermediate] What are the averages for that priority example?

WT in execution order is 0,1,6,16,18, totaling 41: average 8.2. TAT is 1,6,16,18,19, totaling 60: average 12.

Q098 [Intermediate] How should equal priorities be handled?

Apply an explicit secondary rule, commonly FCFS. The included stable insertion sort preserves input order for equal priorities when all arrivals are zero.

Q099 [Intermediate] Distinguish static and dynamic priority.

Static priority stays fixed in the model. Dynamic priority changes with behavior or waiting, for example through aging or feedback scheduling.

Q100 [Intermediate] Why can a high-priority process wait?

It may be blocked on I/O, waiting for a resource, or behind a currently running task in a nonpreemptive policy. High priority does not make an unavailable resource appear.

Q101 [Intermediate] What is priority inversion?

A high-priority task waits for a resource held by a lower-priority task. Medium-priority work can further delay the holder and lengthen the high-priority task's wait.

Q102 [Intermediate] What is priority inheritance?

Temporarily boosting a lock holder's effective priority to that of a higher-priority waiter, so it can finish the critical work and release the resource sooner.

Q103 [Expert] Is aging the same as priority inheritance?

No. Aging addresses long waits and starvation; inheritance addresses resource-related priority inversion. They use different triggers and solve different problems.

Q104 [Expert] Why can priority scheduling hurt average waiting time?

Priorities may favor long or important jobs over short ones. It optimizes priority policy rather than necessarily minimizing the mean waiting metric.

Q105 [Expert] Why is sorting all jobs once insufficient for preemptive priority with arrivals?

Eligibility and priorities may change over time. The simulator must reconsider ready work at relevant events and save remaining service when preempting.

08 | Contiguous allocation and fragmentation

Q106 [Basic] What is contiguous memory allocation?

Assigning a process one continuous address range in the modeled memory. The allocator must find a large enough contiguous region.

Q107 [Basic] What is a hole?

A free contiguous region available for allocation. Multiple separated holes cannot automatically satisfy one larger contiguous request.

Q108 [Basic] What is internal fragmentation?

Unused space inside an allocated unit, such as a partition larger than the process occupying it.

Q109 [Basic] What is external fragmentation?

Free space split among separated regions so a contiguous request may fail even when total free memory is sufficient.

Q110 [Basic] What is compaction?

Relocating allocated regions to combine scattered free space into larger regions. It requires relocation support and incurs data movement cost.

Q111 [Intermediate] Is all free memory external fragmentation?

No. A remaining single free region is free capacity, not automatically evidence of fragmentation. Explain whether scattered holes prevent a request from fitting.

Q112 [Intermediate] Give an external-fragmentation example.

Two separated holes of 100 and 200 units contain 300 free units, but a contiguous 250-unit request fits neither. Without moving allocations or changing the model, it cannot be placed.

Q113 [Intermediate] What is coalescing?

Merging adjacent free regions when memory is released. It combines existing neighboring holes without relocating allocated processes.

Q114 [Intermediate] How does splitting a hole work?

Allocate the requested amount from a suitable hole and retain its remainder as a smaller free region. The remainder can satisfy later requests if the model permits reuse.

Q115 [Intermediate] What happens in a whole-block allocation model?

One process reserves an entire chosen block. A used flag prevents another allocation in that block, and leftover capacity inside it is internal waste.

Q116 [Intermediate] Why can two correct fit programs produce different answers?

One may reserve whole blocks while another splits and reuses residual holes. Block ordering and tie rules can also differ; compare assumptions before blaming the algorithm.

Q117 [Intermediate] What does allocation failure imply?

No currently eligible block or hole fits under the stated model. It does not necessarily imply every byte is occupied or that smaller later requests must also fail.

Q118 [Expert] Does best fit always eliminate fragmentation?

No. It minimizes the immediate residual among eligible regions, but can leave tiny holes and may perform worse for future requests. Long-term behavior depends on the workload.

Q119 [Expert] Why is compaction different from paging?

Compaction physically moves contiguous allocations to consolidate holes. Paging permits noncontiguous physical placement through address translation, avoiding the same contiguous-allocation requirement.

Q120 [Expert] Can an allocator report total free space and still fail a request?

Yes. It must consider the size of the largest suitable contiguous region, alignment and policy restrictions, not just the sum of free bytes.

09 | Practical: First, best and worst fit

Q121 [Basic] What is first fit?

Scan regions in the defined order and choose the first one large enough for the request.

Q122 [Basic] What is best fit?

Choose the smallest eligible region large enough for the request, equivalently minimizing the nonnegative immediate leftover.

Q123 [Basic] What is worst fit?

Choose the largest eligible region, aiming to leave a relatively large remainder rather than a tiny one.

Q124 [Basic] What does the used-block flag mean in the supplied code?

It marks a block already reserved for a process. When that flag is used without splitting, the remaining capacity is not available for another process.

Q125 [Basic] What should represent 'not allocated'?

An explicit invalid index such as -1, checked before indexing the block array. A valid block number must never double as a failure marker.

Q126 [Intermediate] How does first fit behave on blocks 100,500,200,300,600 and requests 212,417,112,426?

In whole-block mode: P1->B2, P2->B5, P3->B3, P4 fails. In split-hole mode P3 reuses B2's remainder; P4 still fails.

Q127 [Intermediate] How does best fit behave on that example?

Both modes allocate P1->B4, P2->B2, P3->B3 and P4->B5. Their leftover values match immediately, but only split mode keeps those remainders available.

Q128 [Intermediate] How does worst fit behave on that example?

Whole-block mode: P1->B5, P2->B2, P3->B4, P4 fails. Split mode: P1->B5, P2->B2, P3 reuses B5, P4 fails.

Q129 [Intermediate] Why does first fit stop its scan early?

Once the first suitable region is found, later regions cannot change the first-fit choice. Best and worst fit ordinarily need to examine all candidates in an unsorted list.

Q130 [Intermediate] What is the simple array implementation's complexity?

Up to O(processes x blocks). First fit may stop early but still has that worst-case bound; more sophisticated indexing can change performance.

Q131 [Intermediate] Why is a fixed best-remainder sentinel such as 9999 unsafe?

A valid region with leftover 9999 or more may never be selected. Use chosen=-1 and compare candidate sizes only after a candidate exists, or use a justified bound.

Q132 [Intermediate] What should happen after one request fails?

Report that request as unallocated. Continue checking later requests if the task permits, because a smaller later request may fit. Do not silently suppress its result.

Q133 [Expert] Why is the supplied best-fit print loop problematic?

It stops when parray[i]==0, so one failed allocation can hide later results. A per-process allocation status and a complete reporting loop avoid that omission.

Q134 [Expert] What does '--split' change in the included C programs?

It subtracts a request from the chosen hole and allows its remainder to be reused. Without the option the program matches the whole-block flags model. Neither mode includes deallocation or coalescing.

Q135 [Expert] What must you say if the examiner's expected mapping differs from yours?

State region order, whole-block versus split-hole behavior and tie rule. Then trace the first differing allocation; this usually identifies the model mismatch.

10 | Practical: MFT and MVT

Q136 [Basic] Expand MFT in this syllabus.

Multiprogramming with a Fixed number of Tasks. The lab models fixed-size memory partitions, with at most one allocated process per partition.

Q137 [Basic] Expand MVT in this syllabus.

Multiprogramming with a Variable number of Tasks. Partitions are sized dynamically to process requests; the supplied program simplifies this to subtraction from remaining memory.

Q138 [Basic] What fragmentation is associated with fixed partitions?

Internal fragmentation when a process occupies less than its reserved partition. There can also be an unused tail if total memory is not a multiple of partition size.

Q139 [Basic] What fragmentation is associated with variable contiguous partitions?

External fragmentation can arise as allocations are freed and holes form. A simple no-freeing subtraction program does not simulate those scattered holes.

Q140 [Basic] How many equal fixed partitions fit?

floor(total memory / partition size). The unpartitioned tail is total memory modulo partition size.

Q141 [Intermediate] Solve MFT for total 1000 and partition size 300.

There are three complete partitions and a 100-unit tail. The tail is reported separately from unused space inside allocated partitions.

Q142 [Intermediate] Allocate requests 275,400,290,293,100 in that MFT example.

P1, P3 and P4 occupy the three partitions. P2 is too large; P5 has no partition left. Internal waste is 25+10+7=42.

Q143 [Intermediate] Is an unused whole partition internal fragmentation?

Not of an allocated process. It is unallocated capacity. Distinguish unused partitions, unpartitioned tail and internal waste in occupied partitions.

Q144 [Intermediate] Trace MVT pool 1000 with requests 400,275,550.

Allocate 400, leaving 600; allocate 275, leaving 325; reject 550. Total allocated is 675 and free remaining is 325.

Q145 [Intermediate] Why is 'memory is full' imprecise for the failed 550-unit request?

There are still 325 free units. The request is too large for the remaining capacity, but a smaller request could fit.

Q146 [Intermediate] How does the included MVT program differ from the supplied loop?

It takes a bounded number of requests and continues after a failure. It calls remaining capacity 'free memory' rather than automatically labelling it external fragmentation.

Q147 [Intermediate] What additional structures are needed for realistic MVT?

Track allocated regions and a free-hole list with addresses and sizes; implement release, adjacency checks, coalescing and placement policy. A single remaining-memory integer is insufficient.

Q148 [Expert] Can compaction solve a request larger than total free memory?

No. It can combine scattered free regions but cannot create additional capacity. A request exceeding total available space still cannot fit.

Q149 [Expert] Why can MFT reject a job when plenty of total memory is unused?

A process cannot span partitions in this model. If its request exceeds the largest permitted partition, summing several unused partitions does not help.

Q150 [Expert] Does MVT mean there is no internal waste in real systems?

The ideal lab model assigns exactly the requested amount. Real allocators can introduce alignment, metadata and rounding overhead, so the statement requires that idealization.

11 | Paging and page faults

Q151 [Basic] What is a page?

A fixed-size unit of a virtual address space. Paging maps virtual pages onto physical memory frames.

Q152 [Basic] What is a frame?

A physical-memory unit with the same size as a page in the simple paging model. Any compatible free frame can hold a page.

Q153 [Basic] What is a page table?

A structure mapping virtual page numbers to physical frame information and attributes such as validity and protection.

Q154 [Basic] What is a page fault?

An exception caused when a memory reference requires page-related handling, commonly because a valid virtual page is not resident. Invalid or prohibited accesses require different handling from demand loading.

Q155 [Basic] What is a page hit in the replacement exercise?

The referenced page is already in the simulated frame set, so no page load or replacement is needed for that reference.

Q156 [Intermediate] What is a reference string?

The ordered sequence of virtual page identifiers accessed by a process or workload. The replacement simulation processes it from left to right.

Q157 [Intermediate] Does filling an empty frame count as a fault?

Yes. The referenced page was absent. Replacement is unnecessary, but the miss is still counted in the lab fault total.

Q158 [Intermediate] When is replacement required?

When a needed page is absent and no eligible free frame remains. Choose a victim using the policy, preserving dirty data as necessary in a real system.

Q159 [Intermediate] What is a dirty page?

A page modified since its backing copy was last synchronized. Evicting it may require a write-back; a clean page may be discardable without that write.

Q160 [Intermediate] What is the page-fault ratio?

Number of faults divided by number of references. Hit ratio is hits divided by references, and their sum is one in this simplified two-outcome model.

Q161 [Intermediate] Why initialize frame entries?

The simulator must distinguish empty slots from resident pages. An uninitialized array can falsely report hits or choose invalid victims.

Q162 [Intermediate] Why is page number zero valid here?

Page IDs are nonnegative and zero is a legitimate identifier. The included programs use -1 only internally for an empty frame and print '-' for that state.

Q163 [Expert] Is a page fault always a disk operation?

No. It may be resolved by zero-fill, a page already cached in memory, copy-on-write or another in-memory action. Some faults do require storage I/O.

Q164 [Expert] Does paging eliminate every kind of fragmentation?

It avoids the need for a process's physical allocation to be contiguous, but can waste space within its final page and adds page-table overhead.

Q165 [Expert] Why can minimizing faults alone be insufficient for performance?

Victim write-back cost, access latency, CPU overhead, prefetching and interference also matter. The lab compares fault counts under equal simplified costs.

12 | Practical: FIFO page replacement

Q166 [Basic] What does FIFO replace?

The page that has been resident longest since its most recent loading, regardless of how recently it was accessed.

Q167 [Basic] What data structure naturally represents FIFO?

A queue of resident pages. A fixed frame array with a circular replacement pointer is a simple equivalent implementation.

Q168 [Basic] Does a hit move the FIFO pointer?

No. A hit does not reload the page or change its insertion age in ordinary FIFO.

Q169 [Basic] How does a circular replacement pointer advance?

next=(next+1)%frame_count after inserting on a fault. Frame count must be positive to avoid invalid modulo operation.

Q170 [Basic] What is Belady's anomaly?

For some reference strings, increasing frame count makes FIFO incur more page faults instead of fewer.

Q171 [Intermediate] Trace 1,2,1,3 with two initially empty frames.

1 faults, 2 faults, 1 hits, then 3 replaces 1 because it was loaded first. Total faults are three; the hit on 1 does not make it newer in FIFO.

Q172 [Intermediate] What is FIFO's result for the standard 20-reference lab string with three frames?

For 7,0,1,2,0,3,0,4,2,3,0,3,2,1,2,0,1,7,0,1, FIFO has 15 faults and 5 hits.

Q173 [Intermediate] What are FIFO faults for only the first ten references of that string?

Nine faults. The fifth reference, page 0, is the only hit among those first ten references under three-frame FIFO.

Q174 [Intermediate] Give the standard anomaly example.

For 1,2,3,4,1,2,5,1,2,3,4,5, FIFO produces 9 faults with three frames and 10 faults with four frames, starting empty.

Q175 [Intermediate] What changes on a fault when frames are not yet full?

The next empty frame is filled and the insertion order advances. Do not count a fictitious evicted page while filling initial free space.

Q176 [Intermediate] Why print frame contents on hits as well?

It makes the trace auditable and shows that state did not change. The supplied FIFO program prints blank frame rows on hits, which can confuse manual checking.

Q177 [Intermediate] What is the simple FIFO simulator complexity?

O(reference_count x frame_count), because each reference searches all frames for a hit. Pointer replacement itself is O(1).

Q178 [Expert] Why does a hit not protect a frequently used page in FIFO?

FIFO records loading age rather than access recency or frequency. A heavily used old page can still be next for eviction.

Q179 [Expert] Does every increase in FIFO frames cause the anomaly?

No. It is possible for particular strings and frame counts, not an inevitable result. Trace the same reference sequence under both capacities.

Q180 [Expert] Why can a wrong pointer update still pass a simple test?

A sequence with no repeated references gives no hit case to expose the bug. Include a hit immediately before a later miss when testing FIFO behavior.

13 | Practical: LRU page replacement

Q181 [Basic] What does LRU stand for?

Least Recently Used. It evicts the resident page whose latest access is furthest in the past.

Q182 [Basic] What information does timestamp-based LRU keep?

The most recent reference index or access time for each resident page. Smaller timestamps represent older last use.

Q183 [Basic] Must LRU update a timestamp on a hit?

Yes. A hit is a new access and changes recency, even though the page does not move to a different frame.

Q184 [Basic] What happens on a miss with an empty frame?

Fill an empty frame and record the current access time. Victim selection among resident pages is unnecessary until frames are full.

Q185 [Basic] How does LRU differ from FIFO?

FIFO uses time of loading; LRU uses time of most recent access. Hits affect LRU's recency but not FIFO's insertion order.

Q186 [Intermediate] Trace 1,2,1,3 with two frames under LRU.

Load 1, load 2, hit 1, then replace 2 with 3 because 2 is least recently used. Three faults occur, but the victim differs from FIFO.

Q187 [Intermediate] What is LRU's result for the standard 20-reference string and three frames?

It produces 12 faults and 8 hits. Recompute timestamps on every reference to reproduce that result.

Q188 [Intermediate] How can a stack represent exact LRU?

Keep pages ordered by recency and move the referenced page to the most-recent position. Evict the least-recent resident page when required.

Q189 [Intermediate] What is locality of reference?

Programs often reuse recently accessed data and nearby addresses. Temporal locality helps explain why recency-based policies can perform well.

Q190 [Intermediate] What is the simple timestamp LRU complexity?

O(R x F) for R references and F frames: search for a hit and, on a full miss, scan for the oldest timestamp.

Q191 [Intermediate] How does LRU break a timestamp tie?

The classroom implementation uses the first frame encountered. With unique reference indices, distinct occupied pages normally have distinct latest-access times.

Q192 [Intermediate] What bug makes LRU behave closer to FIFO?

Updating the age only when a page is loaded. Correct LRU updates latest use on every hit as well as on insertion.

Q193 [Expert] Why does LRU avoid Belady's anomaly?

Under the standard model it has the stack property: the resident set for k frames is contained in that for k+1 frames. More frames therefore cannot increase misses for the same reference sequence.

Q194 [Expert] Why is exact LRU expensive in a real OS?

Every memory access would need to maintain exact ordering or time information. Hardware support and software overhead motivate approximations such as Clock.

Q195 [Expert] Is LRU always as good as Optimal?

No. LRU infers future usefulness from past activity; Optimal uses the actual future sequence. A pattern that defeats recency can give LRU more faults.

14 | Practical: Optimal replacement

Q196 [Basic] What does Optimal replace?

The resident page whose next use lies furthest in the future, or one that will never be used again.

Q197 [Basic] Why is it called Optimal?

For a known fixed reference sequence and frame count in the standard model, it minimizes the number of page faults.

Q198 [Basic] Why is it not directly implementable online?

A real OS generally does not know the exact future reference string. The lab can look ahead because the whole sequence is provided as input.

Q199 [Basic] What is its practical role?

A benchmark for comparing feasible policies and a useful way to understand the best achievable fault count under the model.

Q200 [Basic] What if a page is never referenced again?

It is an ideal victim. If several pages are never reused, any one of them may be selected without increasing the optimal fault total.

Q201 [Intermediate] What is Optimal's result for the standard 20-reference string and three frames?

Nine faults and eleven hits. Victim frame layouts can differ when there are ties, while the minimum fault count remains the same.

Q202 [Intermediate] How is the next-use index found?

For each resident page, scan forward from the next reference until it appears. Use an explicit beyond-end value when it never appears again.

Q203 [Intermediate] Should Optimal replace a page on a hit?

No. The requested page is already resident. Lookahead victim selection is needed only on a miss with no free frame.

Q204 [Intermediate] How are empty frames handled?

Fill an empty frame before evicting any resident page. A faulty implementation that evicts too early can waste available capacity.

Q205 [Intermediate] For frames [1,2,3] and future references 2,1,4,3, which victim is best now?

Page 3 is used latest among those residents. If a new page must be inserted before that future sequence, replace 3.

Q206 [Intermediate] Why must next-use information be reset for each replacement?

The remaining future sequence changes at every step. Stale positions can choose a page based on a future access that has already occurred.

Q207 [Intermediate] What is the naive lookahead simulator complexity?

Up to O(R squared x F), because each full miss may scan much of the remaining reference string for each frame. More advanced preprocessing can improve this.

Q208 [Expert] Can Optimal have more faults than FIFO on the same fixed model and input?

No. If a reported Optimal count is higher, check the implementation, initial contents, frame count or reference string; the comparisons may not match.

Q209 [Expert] Does a different tie victim prove one Optimal trace is wrong?

No. If multiple resident pages have no future use, several frame states can be equally optimal. Compare declared tie rules and final fault counts.

Q210 [Expert] Is least frequently used the same as Optimal?

No. Frequency counts past accesses, while Optimal considers next future use. A historically frequent page might never be needed again.

15 | Deadlock fundamentals

Q211 [Basic] What is deadlock?

A set of processes cannot proceed because each waits for an event or resource release that another process in the set must cause.

Q212 [Basic] Name the four Coffman conditions.

Mutual exclusion, hold and wait, no preemption of the relevant resources, and circular wait. All are necessary for the classic resource deadlock situation.

Q213 [Basic] What is hold and wait?

A process holds one or more resources while waiting to acquire additional resources.

Q214 [Basic] What does no preemption mean in deadlock theory?

A resource cannot be forcibly taken away safely; it must be released by its holder. This is different from preempting CPU execution.

Q215 [Basic] What is circular wait?

A cycle of processes each waiting for a resource held by the next process in the cycle.

Q216 [Intermediate] Distinguish deadlock and starvation.

In deadlock, involved processes mutually block progress. In starvation, a process is indefinitely denied service while other work can continue.

Q217 [Intermediate] What is deadlock prevention?

Designing rules that break at least one necessary condition, such as imposing a global resource acquisition order to prevent circular wait.

Q218 [Intermediate] What is deadlock avoidance?

Examining prospective allocations and allowing only those that retain a safe state under declared future claims. Banker's algorithm is the main lab example.

Q219 [Intermediate] What is deadlock detection?

Allowing allocations without guaranteeing safety, then checking whether a deadlock has occurred and taking recovery action.

Q220 [Intermediate] What is a safe state?

A state with at least one ordering in which every process can obtain its remaining declared resources and finish, with completed processes releasing their holdings.

Q221 [Intermediate] Is every unsafe state already deadlocked?

No. Unsafe means the model cannot guarantee completion for all possible requests up to the declared maxima. Actual requests may still permit progress.

Q222 [Intermediate] Does a cycle in a resource-allocation graph always prove deadlock?

With one instance of each resource type, a cycle is sufficient. With multiple instances, a cycle alone is not generally sufficient.

Q223 [Expert] How does lock ordering prevent circular wait?

Require every thread to acquire locks in the same global order. A cycle would require contradictory increasing order, so the circular-wait condition cannot form under compliant use.

Q224 [Expert] What are costs of deadlock recovery?

Terminating tasks or preempting resources can lose work, corrupt partially updated state or require rollback. Recovery must respect resource semantics.

Q225 [Expert] Is an OS's ability to preempt a CPU enough to prevent all deadlocks?

No. A preempted process can still hold locks or other resources. CPU scheduling does not automatically release resources safely.

16 | Practical: Banker's algorithm

Q226 [Basic] What does Available contain?

Currently unallocated instances of each resource type. It is a vector, not the total inventory including resources already held.

Q227 [Basic] What do Max and Allocation contain?

Max is each process's declared maximum claim per resource type; Allocation is what it currently holds. Both are process-by-resource matrices.

Q228 [Basic] How is Need calculated?

Need[i][j] = Max[i][j] - Allocation[i][j]. A negative result means the supplied state violates the declared maximum and should be rejected.

Q229 [Basic] What is a safe sequence?

An ordering of processes that can all hypothetically finish using currently free resources plus holdings released by earlier finishes.

Q230 [Basic] What is Work initialized to?

A copy of Available. Finish flags are initially false. The safety test changes Work without pretending the real processes actually executed.

Q231 [Intermediate] When can a process be chosen in the safety test?

When it is unfinished and Need[i][j] <= Work[j] for every resource type j. A scalar sum comparison is insufficient.

Q232 [Intermediate] What is added to Work when a process hypothetically finishes?

Its current Allocation, not its Max. The additional Need would be borrowed and returned within the hypothetical execution, leaving a net gain equal to Allocation.

Q233 [Intermediate] When is the state declared unsafe?

When unfinished processes remain and a full scan finds none whose Need fits Work. No complete safe sequence was found under the model.

Q234 [Intermediate] What are the first checks for a request?

Require nonnegative Request, Request <= Need for the requester, and Request <= Available, component by component. Exceeding Need is invalid; unavailable resources require waiting.

Q235 [Intermediate] How is a tentative request applied?

Available decreases by Request; the requester's Allocation increases by it; Need decreases correspondingly. Run the safety test on that tentative state.

Q236 [Intermediate] What happens if the tentative state is unsafe?

Do not grant the request. Restore the original state and make the requester wait, rather than committing an allocation that removes the safety guarantee.

Q237 [Intermediate] Is the safe sequence unique?

Usually not. Several processes may be eligible at a step. The included code scans increasing process IDs, so its returned sequence is deterministic for the given input.

Q238 [Expert] Why is Need <= Work a componentwise test?

Resource types are not interchangeable. Free printers cannot satisfy a request for memory simply because total instance counts happen to match.

Q239 [Expert] What assumptions limit Banker's practical use?

Processes must state credible maximum claims, the inventory model must be known, and completion must release resources. Dynamic behavior and checking cost can make these assumptions inconvenient.

Q240 [Expert] Does printing a safe sequence implement the full request algorithm?

No. That only demonstrates the safety check. A complete request demonstration also validates a request, tries it, reruns safety, and grants or rolls back the change.

17 | Producer-consumer and synchronization

Q241 [Basic] What is a race condition?

Correctness depends on the timing or ordering of concurrent accesses to shared state without sufficient synchronization.

Q242 [Basic] What is a critical section?

Code that accesses shared state requiring controlled concurrent access. The buffer, indices and counters in this practical are protected by a mutex.

Q243 [Basic] What is the bounded producer-consumer problem?

Producers insert items into a finite buffer and consumers remove them. Synchronization must prevent overflow, underflow and conflicting buffer updates.

Q244 [Basic] What is a semaphore?

A synchronization object manipulated by atomic wait and signal operations. Counting semaphores can track available buffer slots and available items.

Q245 [Basic] What is a mutex?

A lock that provides exclusive ownership of a critical section. It protects the shared buffer while a producer or consumer updates it.

Q246 [Intermediate] What are the initial synchronization values for capacity five?

empty=5, full=0 and the mutex is initially unlocked. A textbook semaphore used as the mutex would start at 1.

Q247 [Intermediate] Explain the producer's operation order.

Create an item; wait(empty); lock the mutex; insert at in; advance in modulo capacity; unlock the mutex; signal(full). Never insert before reserving an empty slot.

Q248 [Intermediate] Explain the consumer's operation order.

wait(full); lock the mutex; remove at out; advance out modulo capacity; unlock the mutex; signal(empty); use the item. Never remove before reserving an available item.

Q249 [Intermediate] Why wait for empty or full before acquiring the mutex?

Waiting while holding the buffer mutex can prevent the other side from changing the condition being awaited, causing deadlock.

Q250 [Intermediate] Why use modulo arithmetic for in and out?

(index+1)%capacity wraps the index to zero at the end, making a circular buffer. Separate counts or semaphores distinguish full from empty when in equals out.

Q251 [Intermediate] Why is x++ not automatically thread-safe?

It involves a read-modify-write operation that can lose an update if performed concurrently without synchronization. A mutex or an appropriate atomic operation is needed.

Q252 [Intermediate] What termination rule is used in the corrected example?

Five producers each insert five unique items and five consumers each remove five items. All 25 items must be consumed exactly once, and main joins all threads before destroying synchronization objects.

Q253 [Expert] Is empty+full always exactly the buffer capacity during execution?

Not necessarily at every instant: a thread can reserve a semaphore unit before publishing the corresponding update to the other semaphore. At quiescent boundaries with no reserved operations the total equals capacity.

Q254 [Expert] Why can mismatched producer and consumer totals hang this finite example?

Extra consumers can wait forever for nonexistent items, or excess production can leave producers blocked on a full buffer. General designs use a coordinated shutdown protocol or termination markers.

Q255 [Expert] Does a successful test prove every possible concurrent execution correct?

No. Repeated runs exercise only some interleavings. Reason about invariants, operation ordering and synchronization as well as checking observed totals and item uniqueness.

18 | Virtual-memory and address-translation follow-ups

Q256 [Basic] What is virtual memory?

An abstraction that gives processes address spaces translated and protected by the OS and hardware, allowing physical placement to differ from virtual layout.

Q257 [Basic] What is a virtual address?

An address generated in a process's virtual address space. Translation determines the physical location or raises an exception.

Q258 [Basic] What is a TLB?

A Translation Lookaside Buffer caches address translations and related permissions, reducing repeated page-table lookup cost.

Q259 [Basic] What is demand paging?

Bringing a page into memory when it is needed rather than loading every page in advance.

Q260 [Basic] What is thrashing?

Excessive paging caused when active workloads lack enough resident memory for useful locality, leaving little time for productive execution.

Q261 [Intermediate] Split an address into page number and offset.

For page size P, page number=floor(address/P) and offset=address mod P. With a power-of-two page size, low address bits form the offset.

Q262 [Intermediate] Translate virtual address 2500 with 1024-byte pages if page 2 maps to frame 7.

Page number=2 and offset=452. Physical address=7 x 1024 + 452 = 7620.

Q263 [Intermediate] What is the difference between a TLB miss and a page fault?

A TLB miss means the translation is not cached. The page may still be resident and translated by a page-table walk; absence from physical memory is a different condition.

Q264 [Intermediate] What is temporal locality versus spatial locality?

Temporal locality is reuse of recently accessed items. Spatial locality is access to nearby addresses. Different optimizations exploit each pattern.

Q265 [Intermediate] What is a working set?

The pages used over a recent execution interval. Keeping active working sets resident helps reduce excessive faults.

Q266 [Intermediate] Distinguish local and global replacement.

Local replacement chooses among a process's allocated frames. Global replacement can choose frames across processes, allowing one workload to affect another's residency.

Q267 [Intermediate] What is copy-on-write?

Processes initially share read-only physical copies; when one writes, the system creates a private copy as needed. This avoids unnecessary eager copying.

Q268 [Expert] Can increasing multiprogramming worsen throughput?

Yes. If active working sets exceed memory, increased paging and contention can outweigh the benefit of more runnable work and lead to thrashing.

Q269 [Expert] Why can larger pages have mixed effects?

They reduce the number of page-table entries and can improve translation coverage, but may increase internal waste and transfer unnecessary data. Workload locality matters.

Q270 [Expert] What is the cost of a page table for a 32-bit byte address space with 4 KiB pages and 4-byte entries?

There are 2^20 virtual pages, so a flat page table has 2^20 x 4 bytes = 4 MiB, before extra metadata. Multi-level designs can avoid allocating unused portions.

19 | Dining Philosophers

Q271 [Basic] What does Dining Philosophers demonstrate?

Several concurrent participants need pairs of shared resources. It illustrates mutual exclusion, deadlock and starvation in resource allocation.

Q272 [Basic] Why are five forks shared by five philosophers?

Each fork lies between two neighbors, and each philosopher needs both adjacent forks to eat. Resources can conflict even though participants have separate execution streams.

Q273 [Basic] What states are used in the supplied approach?

THINKING, HUNGRY and EATING. HUNGRY records a request; EATING records that both neighboring resources have been reserved.

Q274 [Basic] Can neighboring philosophers eat simultaneously?

No, because they share a fork. Nonadjacent philosophers can eat simultaneously when their other neighbors do not conflict.

Q275 [Basic] What does the global mutex protect?

Shared state checks and state changes, including deciding whether a hungry philosopher may enter EATING. It is not intended to remain locked throughout eating.

Q276 [Intermediate] Why can picking up the left fork first cause deadlock?

If everyone holds one fork while waiting for the next, a circular wait forms. Merely swapping names left and right for everyone does not solve the problem.

Q277 [Intermediate] Explain the test(i) operation.

While holding the mutex, check that i is HUNGRY and neither neighbor is EATING. If so, change i to EATING and signal its private semaphore.

Q278 [Intermediate] Explain take_forks in the state-based algorithm.

Lock the mutex, set HUNGRY, call test(i), unlock the mutex, then wait on i's private semaphore. Waiting outside the mutex allows neighbors to finish and grant permission.

Q279 [Intermediate] Explain put_forks in the state-based algorithm.

Lock the mutex, set the current state to THINKING, test the left and right neighbors, then unlock. The release can make an adjacent hungry philosopher eligible.

Q280 [Intermediate] Why initialize every private semaphore to zero?

A philosopher must obtain permission before eating. test(i) posts exactly when it reserves the forks for that philosopher.

Q281 [Intermediate] Is a signal lost if it occurs before sem_wait?

No. The semaphore retains a count that the later wait can consume. This differs from incorrectly using an unprotected transient flag.

Q282 [Intermediate] How many philosophers can eat at once with five arranged in a ring?

At most two, because no two adjacent philosophers may eat. The example may show less concurrency depending on thread scheduling and how short the simulated work is.

Q283 [Expert] Is the state-based solution guaranteed starvation-free?

No fairness queue or bounded-wait rule is implemented. It prevents adjacent eating and avoids the circular hold-one-fork deadlock, but indefinite overtaking is possible in an ongoing workload without stronger fairness assumptions.

Q284 [Expert] Give two alternative deadlock-prevention designs.

Acquire forks by a global numeric order, removing circular wait; or admit at most four of five philosophers to the fork-acquisition phase. Each needs correct release and separate consideration of fairness.

Q285 [Expert] Why avoid sleep while holding the global mutex?

It needlessly prevents other philosophers from checking or releasing their states. Sleeping is not synchronization; any simulated eating delay belongs outside the short state-management critical section.

20 | C code, debugging and practical defense

Q286 [Basic] Why is stdio.h included?

It declares standard input/output functions such as printf and scanf so the compiler can check their use correctly.

Q287 [Basic] Why does scanf usually need & for an integer variable?

It must receive the address where the converted value is stored. Passing the integer's current value instead is a type and memory-access error.

Q288 [Basic] What are valid indices of int a[20]?

0 through 19. Index 20 is out of bounds. One-based display labels do not require one-based array storage.

Q289 [Basic] What does break do versus continue?

break exits the nearest loop or switch; continue skips the remainder of the current loop iteration and proceeds to the loop's next iteration step.

Q290 [Basic] Why initialize accumulators and flags?

Automatic local variables otherwise contain indeterminate values. Averages, allocation checks and completion tests can fail unpredictably if they start from garbage state.

Q291 [Intermediate] Why preserve original burst time in a preemptive program?

Remaining time is reduced during simulation, but WT=TAT-BT needs the original CPU service. Overwriting the only copy makes the final metrics wrong.

Q292 [Intermediate] Why validate process count and frame count?

Prevent array overruns, division by zero, modulo by zero and meaningless empty simulations. The allowed count must match the actual declared capacity and indexing convention.

Q293 [Intermediate] Why check input conversion?

Invalid text or missing data can leave values unchanged or uninitialized. The included programs parse bounded integer tokens and reject malformed or out-of-range input.

Q294 [Intermediate] What is the issue with sum/n when both are integers?

Integer division discards the fractional part before assignment to a floating variable. Use a floating accumulator or convert an operand before dividing.

Q295 [Intermediate] Why avoid conio.h, clrscr and getch in portable C?

They are not part of standard C and may be unavailable in the exam compiler. Standard input/output is enough for these algorithms.

Q296 [Intermediate] What is the difference between compile-time and logical errors?

Compile-time errors prevent a valid build, while logical errors can produce wrong results in a program that compiles. Compiler warnings and known-answer tests address different problems.

Q297 [Intermediate] What should you explain when defending the program?

State input meaning, assumptions, main arrays, selection rule, update rule, stopping condition, output calculation and one edge case. Connect each important loop to the algorithm.

Q298 [Expert] Why is static zero initialization not a complete allocation-failure solution?

A zero index can still refer to a valid element or mask a missing result. Use explicit success/failure state and avoid indexing until a candidate was actually selected.

Q299 [Expert] What tests are stronger than repeating the sample input?

Include ties, repeated hits, exact fits, failed requests followed by smaller ones, idle CPU gaps, invalid counts and unsafe Banker trials. Compare against independent calculations or a separate reference model.

Q300 [Expert] What is the best final check before demonstrating an experiment?

Compile the exact saved source, run a known case, verify the full trace and averages, then explain the assumptions. Do not rely on an old executable that may no longer match the source.

Last-minute revision | Scheduling and memory

Item	Remember
TAT	CT - AT
WT	TAT - BT for one CPU burst with no I/O waiting
RT	First start - AT
FCFS	Arrival order; convoy effect; nonpreemptive
SJF / SRTF	Shortest ready burst / shortest remaining burst
RR	Positive quantum; incomplete jobs return to ready-queue tail
Priority	Declare direction, preemption and tie rule; aging addresses starvation
First / Best / Worst Fit	First adequate / smallest adequate / largest adequate hole
Internal fragmentation	Unused space inside an allocated unit
External fragmentation	Separated holes cannot satisfy a contiguous request despite sufficient total space
MFT	Fixed partitions; process must fit one; partition count limits admission
MVT model here	Variable request sizes deducted from a single pool; no free/coalesce simulation

Say the assumption before the answer

All arrivals zero? One CPU burst? Zero context-switch cost? Lower number means higher priority? Whole blocks or split holes? Empty initial frames? Known maximum claims? These details determine whether your numerical answer is valid.

Last-minute revision | Paging and concurrency

Item	Remember
FIFO	Oldest insertion; hits do not renew age; anomaly possible
LRU	Oldest last access; hits DO update recency
Optimal	Farthest next use; needs future reference knowledge
Deadlock conditions	Mutual exclusion, hold-and-wait, no preemption, circular wait
Prevention	Break a necessary condition; global resource ordering breaks circular wait
Avoidance	Stay in safe states; Banker needs maximum claims
Banker safety	Need=Max-Allocation; Work starts Available; eligible if Need<=Work; release Allocation
Request algorithm	Check Need; check Available; tentatively allocate; safety-test; grant or rollback
Producer	wait(empty) -> lock -> insert -> unlock -> post(full)
Consumer	wait(full) -> lock -> remove -> unlock -> post(empty)
Dining Philosophers	Neighbor exclusion; atomically reserve both forks; wait outside mutex
Liveness distinctions	Deadlock: mutual blocking. Starvation: one continually denied service. Livelock: activity without useful progress.

Final mock viva

Pick any two practical cards. Without notes: state the aim in one sentence, draw the data structures, trace a fresh small input, identify one failure case, and explain one line you would change for a different assumption. If you can do this clearly, you are preparing beyond memorized definitions.

Sources and verification

Primary scope: your two ZIP archives, 3-2. OS Lab and 3. OS, plus the 48-page scanned 3. OS.pdf. The lab-list PDF establishes the 12 core topics; the older archive and handwritten report establish the five additional topics. Source PDFs were text-extracted or OCR-read; the corrected implementations were authored separately.

The source material contains variations and mistakes. This guide identifies the important differences instead of treating every screenshot or sample program as authoritative. The supplemental virtual-memory and process questions are general OS background, not additional claimed lab assignments.

Further reading

Operating Systems: Three Easy Pieces, author site

OSTEP: CPU scheduling

OSTEP: page replacement policies

Linux manual: pthread_create

Linux manual: sem_wait

What was tested

All 17 programs compiled with strict C11 warnings treated as errors. The first 14 passed 1,134 process executions, including independent reference comparisons for scheduling, allocation and page traces, small Banker states checked by exhaustive process orders, request rollback and invalid-input checks. The three threaded programs passed 150 additional executions with completion and shared-state assertions.

Repeated thread runs do not cover every possible interleaving and are not a mathematical proof of fairness. The guide therefore also explains invariants and lock ordering. These are educational simulations and small synchronization demonstrations, not replacements for the host operating system.

Practice inputs and outputs are in os-viva-code/examples. Thread traces may vary in order; compare the documented totals and invariants. All question answers are original explanations intended for oral revision.